```
section .data
  hello:     db 'Hello, World!',10    ; 'Hello, World!' plus a linefeed character
  helloLen:  equ $-hello             ; Length of the 'Hello world!' string

section .text
  global _start

_start:
  mov eax,4            ; The system call for write (sys_write)
  mov ebx,1            ; File descriptor 1 - standard output
  mov ecx,hello        ; Put the offset of hello in ecx
  mov edx,helloLen     ; helloLen is a constant, so we don't need to say
                       ;  mov edx,[helloLen] to get it's actual value
  int 80h              ; Call the kernel
  mov eax,1            ; The system call for exit (sys_exit)
  mov ebx,0            ; Exit with return "code" of 0 (no error)
  int 80h;
```

STDIN

Input for the program ( Optional )

Output: 1 ms |

```
Hello, World!
```

```
section .data
msg db "Computer Architecture & Operating Systems!", 0ah
section .text
global _start
_start:
mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, 42
syscall
mov rax, 60
mov rdi, 0
syscall
```

STDIN

Input for the program ( Optional )

Output: 1 ms

```
Computer Architecture & Operating Systems!
```

```
section .data
msg db "1 2 3 4 5 6 7 8 9", 0
section .text
global _start
_start:
; Print the message
mov eax, 4 ; syscall number for sys_write
mov ebx, 1 ; file descriptor 1 (stdout)
mov ecx, msg ; pointer to the buffer containing the message
mov edx, 18 ; length of the message
int 0x80 ; Use interrupt for 32-bit systems
; Exit the program
mov eax, 1 ; syscall number for sys_exit
xor ebx, ebx ; exit code 0
int 0x80 ; Use interrupt for 32-bit systems
```

STDIN

Input for the program ( Optional )

Output: 1 ms

```
1 2 3 4 5 6 7 8 9
```